Modeling Performance of Transformers on FPGA

Madan Y N
Indian Institute of Technology Bombay
Mumbai, India
200070040@iitb.ac.in

Ashwin Krishnan
TCS Research
Mumbai, India
ashwin.krishnan1@tcs.com

Manoj Nambiar
TCS Research
Mumbai, India
m.nambiar@tcs.com

***Abstract*—The rapid proliferation of transformer models, such as GPT and BERT, has revolutionized the landscape of artificial intelligence applications, extending their reach beyond natural language processing into the realms of vision and beyond. Yet, the pressing challenge lies in deploying these sophisticated models on edge computing devices, where real-time, low-power, and low-latency processing is paramount. Although FPGAs are good candidates for low-power applications, the development cost in terms of the number of hours required for coding and deploying is huge compared to that of a CPU or a GPU. In this paper with GPT-2 as a reference model, we present a modeling technique that provides an optimal deployment strategy for a given FPGA board enabling quick comparison with CPU and GPU. We investigate two radically different designs, one model-independent and the other specific to the model we are deploying, and compare the performance of the two. We present our modeling and latency formulation techniques along with design space exploration techniques to determine the optimal partitioning of resources. In terms of latency, our model-agnostic design performed 21 times, and the model-specific design performed 119 times better than a dual-core i5 CPU.**

***Index Terms*—constrained optimization, FPGA, GPT-2, latency, transformer**

I. Introduction

The influence of transformer models in natural language processing and beyond is undeniable. From language models like GPT, and BERT to vision transformers, they have redefined the landscape of AI applications. However, as the demand for real-time, low-power, and low-latency processing surges, the challenge lies in their efficient deployment on edge computing devices. This paper embarks on an exploratory journey, using GPT-2 as a reference point, to assess the feasibility of deploying transformer models on such low-power devices for low-latency applications. Several implementations of transformer models in FPGA exist, such as [1], which leverages BCM (block-circulant matrix) techniques for model compression and accelerated computations. However, a limitation of this approach is the necessity to apply BCM compression during the training phase. In contrast, other implementations like [2] and [3] have adopted a versatile modular design encompassing multiple computation units of common operations found in transformer models, including matrix multiplication, softmax, and normalization which are scheduled as required. While this design allows the deployment of various language models without design alterations, it does introduce performance trade-offs. In contrast, our primary aim is to develop an end-to-end pipelined design tailored specifically for GPT-2, optimizing it for this particular application.

II. Methodology

A. GPT-2

GPT-2, or Generative Pre-trained Transformer-2 is a decoder-only large language model (LLM) developed by OpenAI [4]. It utilizes an embedding dimension of 768, enabling the representation of words and tokens in a 768-dimensional vector space to capture semantic relationships. The model's vocabulary encompasses 50,257 tokens, facilitating the comprehension of linguistic nuances. GPT-2 operates with a fixed context length, with the specific variant considered here, GPT-2 small, having a context window of 1024 tokens. This model consists of 12 decoder blocks, resulting in a total of about 124 million parameters. Fig. 1 depicts the block diagram of the above-discussed model.

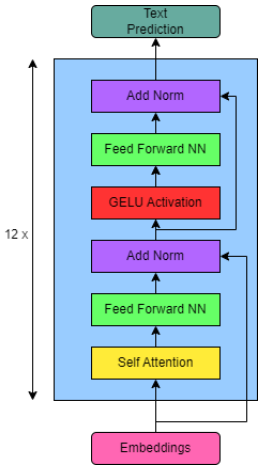


Fig. 1. Functional block diagram of GPT-2

B. Design Approach

Our objective was to develop a low-latency, pipelined architecture to deploy GPT-2 for inference in FPGA. In order to do so, we first identified the compute-intensive functional blocks which form the critical path. These are, self-attention, feed-forward neural network, normalization, and GELU (Gaussian Error Linear Unit) activation function. Comprising all the layers, there are eight matrix multiplication operations, one softmax operation, one $\tanh$ operation (as part of GELU), and

two norm operations in a single decoder block. GPT-2 has 12 such decoder blocks. Subsequently, our focus shifted towards the implementation of circuits for each of the aforementioned mathematical operations, which were then integrated to form a complete decoder block. We leveraged pre-designed components such as adders, multipliers, and functional blocks, including square root and exponential operations to implement the circuits. Notably, this phase marked the introduction of two fundamentally distinct design approaches for implementing the decoder block.

- **Model Agnostic Design (Design-1)** : This is a modular design approach. We make use of the resources available in an FPGA board (DSP slices, LUTs, etc) and implement clusters of matrix multiplication units, softmax units, norm units, etc and dynamically schedule them as required by the LLM. Since the majority of LLMs consist of these operations, it enables users to run numerous models with minimal changes to the core design.
- **Model Specific Design (Design-2)**: Contrary to the previous design, here we implement a circuit specifically for GPT-2. We expect such a design to be faster since there will be efficient usage of resources and there is scope to implement an end-to-end pipelined design.

C. Latency Formulation

The next step is to formulate latency of each of our designs. The fundamental latency expression for total latency L is,

$$L = IL + II * \left\lceil \frac{N-1}{k} \right\rceil \tag{1}$$

IL is iteration latency, latency from the first set of inputs to the first set of outputs, II is initiation interval, the time duration after which the next set of inputs can be fed into the pipeline, N is the total number of inputs and k is the number of inputs fed into the pipeline at a time.

The latency expression for a given design is profoundly influenced by the choice of the scheduling algorithm. Consider, for instance, the self-attention layer, which comprises three matrix multiplication operations—namely, the generation of matrices q, k, and v. It is worth noting that matrices q and k are immediately required for the subsequent matrix multiplication operation, where qk^T is computed, while the matrix v is needed two stages later in the process. Scheduling these three operations in sequence yields a latency expression that differs significantly from a scenario in which computational resources are shared among the three operations. It is really hard to say which will result in a better design just by intuition. We have explored various such scheduling approaches and the latencies provided in the results section are for the best of them.

Fig. 2 is a part of our Design-2 that handles the computation of q and k. The latency formulation for this is as follows

$$IL = T_{mul} + \left(\lceil \log_2 n_1 \rceil + \left\lceil \log_2 \left\lceil \frac{768}{n_1} \right\rceil \right\rceil \right) * T_{add} + \left\lceil \frac{768}{n_1} \right\rceil * T_{buff} \tag{2}$$

T_{mul} is the delay of a floating-point multiplier, T_{add} is the delay of a floating-point adder, T_{buff} is the delay of writing

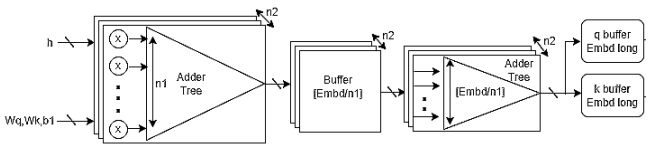


Fig. 2. A part of the datapath

one entry to a buffer, and n_1, n_2 are design parameters. The II for this is 1 given that the adders and multipliers are themselves pipelined, input size N is typically $1024 * 768$ for GPT-2, and k is $n_1 n_2$. Thus total latency is,

$$L = IL + \frac{1024 * 768 - 1}{n_1 n_2} \tag{3}$$

A similar analysis was performed for the whole design to obtain end-to-end latency (not shown due to space constraints).

D. Design Space Exploration

Similar to Fig. 2, our whole design is parameterized. We performed constrained optimization of the design to find the best values for n_1, n_2, and so on using a genetic algorithm. A genetic algorithm is a metaheuristic inspired by natural selection, gene mutation, and gene crossover. Genetic algorithms are known to perform well on nonlinear cost functions, which is desirable for our application. The constraint here is the amount of resources we have at our disposal. Moving back to Fig. 2, the amount of resources R required to implement this part is,

$$R = n_1 n_2 (R_{mul} + R_{add}) + \left\lceil \frac{768}{n_1} \right\rceil n_2 (R_{buff} + R_{add}) \tag{4}$$

R_{add} and R_{mul} are the resources (DSP slices and LUTs) required to implement an adder and a multiplier, respectively, and R_{buff} is the amount of block RAM required for buffers. The total resource usage for the whole design is computed similarly.

III. Results

Table I summarizes the inference latencies for CPU, Design-1, and Design-2. The CPU latency is experimentally measured whereas the other two are computed using the latency expressions. For both designs, we have taken the amount of resources equal to that of Xilinx Alveo-U280 and a clock frequency of 100 MHz.

TABLE I
GPT-2 Avg. Inference Latencies for text generation task

	CPU (dual-core)	**Design-1**	**Design-2**
Latency(s)	37.69	1.75	0.316

IV. Conclusions And Future Work

As machine learning becomes ubiquitous, the need for efficient, low-latency and low-power implementations is paramount. We've described our efforts to deploy GPT-2 on FPGA, with promising simulation results. Next, we'll translate our design into hardware description languages and rigorously assess latency and power consumption.

References

[1] Bingbing Li, Santosh Pandey, Haowen Fang, Yanjun Lyv, Ji Li, Jieyang Chen, Mimi Xie, Lipeng Wan, Hang Liu, Caiwen Ding, "FTRANS: Energy-Efficient Acceleration of Transformers using FPGA," ISLPED '20: Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design, 10 Aug 2020

[2] Xin Yang, Tao Su, "EFA-Trans: An Efficient and Flexible Acceleration Architecture for Transformers," Electronics, Vol. 11, No. 21, Article 3550, Oct 2022

[3] Suyeon Hur, Seongmin Na, Dongup Kwon, Joonsung Kim, Andrew Boutros, Eriko Nurvitadhi, Jangwoo Kim, "A Fast and Flexible FPGA-based Accelerator for Natural Language Processing Neural Networks," ACM Transactions on Architecture and Code Optimization, Vol. 20, No. 1, Article 11, Feb 2023

[4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI Technical Report, Feb 2019